Variables, types and expressions

What = really does, the four basic types, the arithmetic operators and the order Python applies them in.

THE QUESTION

What does `x = x + 1` mean to Python, and in what order does it apply +, * and **?

THE INTUITION

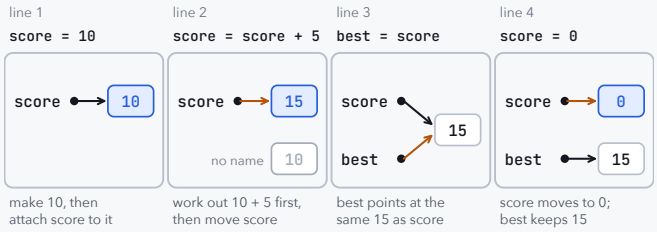
A variable is a name attached to a value. Python works out the right side of = one operator at a time, then moves the name to the result.

KEY TAKEAWAY

= means compute, then attach. / gives a float, // rounds down, `int()` cuts toward zero.

an expression → brackets first → then ** → then * / // % → then + - → one value → = attaches the name

1 = means compute, then attach a name



`print(score, best)` shows `0 15`

blue: value made on this line · amber: arrow moved on this line · grey: no name points here

= is an **assignment**: work out the value on the right, then attach the **variable** (the name) on the left to it. In maths $x = x + 1$ is never true; here `score = score + 5` reads the old 10, makes 15 and moves the name. `best = score` puts a second name on the same value, so moving `score` leaves `best` at 15. Equality is asked with == (see the conditionals sheet).

2 Every value has a type; `type()` shows it

you type	Python's value	type(...) gives
`10`	`10`	`<class 'int'>`
`2.0`	`2.0`	`<class 'float'>`
`"Hello"`	`'Hello'`	`<class 'str'>`
`"10"`	`'10'`	`<class 'str'>`
`True`	`True`	`<class 'bool'>`
`2 ** 70`	`1180591620717411303424`	`<class 'int'>`

quotes make a str: "10" is two characters of text, 10 is a number

A value's **type** decides what you can do with it: **int** (whole numbers, any size), **float** (with a decimal point, even `2.0`), **str** (a **string**: text in quotes) and **bool** (`True` or `False`). A name has no type of its own: after `x = "ten"`, `type(x)` gives `<class 'str'>`.

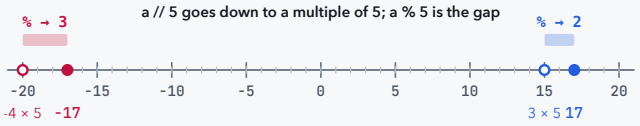
3 Seven arithmetic operators

op	meaning	two ints	a float mixed in
`+`	add	`17 + 5` → `22`	`17.0 + 5` → `22.0`
`-`	subtract	`17 - 5` → `12`	`17.0 - 5` → `12.0`
`*`	multiply	`17 * 5` → `85`	`17.0 * 5` → `85.0`
`/`	divide	`17 / 5` → `3.4`	`17.0 / 5` → `3.4`
`//`	floor divide	`17 // 5` → `3`	`17.0 // 5` → `3.0`
`%`	remainder	`17 % 5` → `2`	`17.0 % 5` → `2.0`
`**`	power	`2 ** 5` → `32`	`2.0 ** 5` → `32.0`

/ always gives a float, even 10 / 2 → 5.0; a float anywhere makes the answer a float

An **expression** is code that has a value; **operators** combine values. Two ints give an int, except with /. // is **floor division** (divide, then round down), % the **remainder**, ** the power.

4 // rounds down, also for negative numbers



a	b	a // b	a % b	check: b * (a // b) + a % b
17	5	3	2	`5 * 3 + 2 = 17`
-17	5	-4	3	`5 * -4 + 3 = -17`
17	-5	-4	-3	`-5 * -4 + -3 = 17`
-17	-5	3	-2	`-5 * 3 + -2 = -17`

`a // b` goes down to a multiple of `b`, so `-17 // 5` is −4, although −17 / 5 is −3.4. `a % b` is the distance back up to `a` and takes the sign of `b`, so the check column always holds. Uses: `n % 2` is 0 exactly when `n` is even; `200 // 60, 200 % 60` gives 3 20: 200 seconds is 3 minutes 20 seconds.

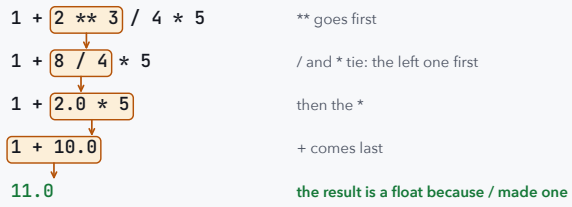
5 Precedence: which operator goes first

`( )`	brackets	`(1 + 2) * 3` → `9`	inside first
`**`	power	`2 ** 3 ** 2` → `512`	right to left
`-x`	negative sign	`-2 ** 2` → `-4`	after **
`* / // %`	multiply, divide	`8 / 4 * 2` → `4.0`	left to right
`+ -`	add, subtract	`10 - 4 - 3` → `3`	left to right

higher rows go first · = runs after all of them: it stores the finished value

Precedence is the order in which Python applies operators, as with BODMAS in school maths. Operators on one row tie and go left to right, except **, which groups right to left: `2 ** 3 ** 2` is `2 ** 9`. ** also beats a minus sign in front: `(-2) ** 2` gives 4.

6 A worked evaluation, step by step



brackets say the same thing out loud: 1 + ((2 ** 3) / 4) * 5 → 11.0

At each step Python applies the operator with the highest precedence; among ties, the leftmost. The answer is `11.0`, a float, because / always makes one. If a reader might hesitate over the order, add brackets.

Change types, update values, choose names

Turning text into numbers and back, what `int()` does to 3.9 and −3.9, shortcuts such as +=, and the rules for names.

7 `int()`, `float()`, `str()` build a value of another type

you call	result	its type
`int("100")`	`100`	`<class 'int'>`
`float("10.56")`	`10.56`	`<class 'float'>`
`str(10)`	`'10'`	`<class 'str'>`
`float(7)`	`7.0`	`<class 'float'>`
`int(float("10.61"))`	`10`	`<class 'int'>`
`int("10.5")`	`ValueError: invalid literal for int() with base 10: '10.5'`	
`int("abc")`	`ValueError: invalid literal for int() with base 10: 'abc'`	

int() of a string needs a whole number written in digits

A **type conversion** makes a new value and leaves the old one alone. `input()` always gives a str (see the input and output sheet), so `int()` turns typed digits into a number. `int("10.5")` is a **ValueError**: right type, wrong value. `len(str(n))` counts the digits of `n`; `len(10)` is a TypeError.

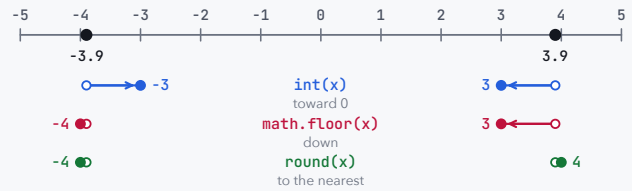
9 `n += 3` is short for `n = n + 3`

line	you write	same as	n after	type
1	`n = 7`		7	`<class 'int'>`
2	`n += 3`	`n = n + 3`	10	`<class 'int'>`
3	`n *= 2`	`n = n * 2`	20	`<class 'int'>`
4	`n //= 4`	`n = n // 4`	5	`<class 'int'>`
5	`n **= 3`	`n = n ** 3`	125	`<class 'int'>`
6	`n %= 7`	`n = n % 7`	6	`<class 'int'>`
7	`n /= 4`	`n = n / 4`	1.5	`<class 'float'>`

each line reads n, works out the new value, then moves n to it; /= makes a float

Augmented assignment updates a name from its own value. Each operator has one: `+=`, `-=`, `*=`, `/=`, `//=`, `%=`, `**=`. The name must exist first: `total += 5` on a new name gives `NameError: name 'total' is not defined`.

8 `int()` cuts toward zero; floor goes down



round() sends a tie to the even neighbour: round(2.5) → 2, round(3.5) → 4

`int()` of a float **truncates**: it drops the part after the point, so `int(-3.9)` is −3. `math.floor` (after `import math`) always goes down, like `-3.9 // 1`, which gives -4.0. Above zero the two agree; below zero they differ by one.

10 Name rules, and what makes a SyntaxError

these names work

- `x1`
- `x_1`
- `first_number`
- `_count`
- `total`
- `Total`

total and Total are two different names: case matters

these are a SyntaxError, so nothing in the file runs

`1x = 2`	`invalid decimal literal`
`for = 5`	`invalid syntax`
`False = 1`	`cannot assign to False`
`my age = 20`	`invalid syntax`
`my-age = 20`	`cannot assign to expression here. Maybe you meant '==' instead of '='?`

Names use letters, digits and _, never start with a digit and are never one of the 35 **keywords** (`help("keywords")` lists them). A **SyntaxError** means Python could not read the file, so not even line 1 runs. Pick names that say what they hold: `first_number`, not `a`.

11 Common mistakes: what you wrote, what Python says or prints, the fix

you wrote	the last line Python prints	why, and the fix
`5 = x`	`SyntaxError: cannot assign to literal here. Maybe you meant '==' instead of '='?`	the name goes on the left: `x = 5`
`x = 3 +`	`SyntaxError: invalid syntax`	finish the expression: `x = 3 + 4`
`age = int("20.5")`	`ValueError: invalid literal for int() with base 10: '20.5'`	go through float: `int(float("20.5"))`
`is_on = true`	`NameError: name 'true' is not defined. Did you mean: 'True'?`	`True` and `False` take a capital
`print(-3 ** 2)`	no error; it prints -9	`**` goes first; `(-3) ** 2` gives 9
`print(7 / 2)`	no error; it prints 3.5	`/` gives a float; `7 // 2` gives 3
`count =+ 1`	no error; it prints 1	`=+ 1` sets 1; from 4, `count += 1` gives 5
`x + 1`	no error; it prints 4	the sum is lost; from 4, `x = x + 1` gives 5

12 Try it (answers in green)

1. Valid names? `2nd` `second_2` `class` `Class`
   `second_2` and `Class`
2. `print(-7 // 2, -7 % 2)`?
   `-4 1`: 2 × −4 + 1 is −7.
3. `print(2 + 3 * 4 ** 2)`?
   `50`: 16, then 48, then 50.
4. `print(int(-2.7), int("42"))`?
   `-2 42`: toward zero.
5. `a = 5, a += 2, a *= a, print(a)`?
   `49`: 7, then 7 × 7.
6. `print(type(10 / 5), 10 / 5)`?
   `<class 'float'> 2.0`

right side first, then attach the name · every value has a type · ** before * / // % before + - · int() cuts toward 0

Source: the variables and expressions notebook, nipunbatra.github.io/comp22. Every output and error message was produced by running the code with Python 3.11.11; other versions word some messages differently. Next: the input, output and basic strings sheet.